6부 전반 — 글자에서 의미로

텍스트 인코딩 · 토큰 · 원-핫에서 임베딩으로 · Word2Vec 미니

앱 페이즈 15~18 (히든 스테이지) · 대상: 고등학생 · 활동 시간 약 90분

학습 목표 ① 글자가 어떻게 숫자가 되는지(코드포인트, UTF-8) 설명한다. ② 토큰의 단위와 한·영 토큰 수 차이의 원인을 설명한다. ③ 원-핫의 한계와 임베딩의 등장 이유를 안다. ④ Word2Vec의 "왕 – 남자 + 여자 = 여왕" 같은 의미 연산을 직접 확인하고 의미를 해석한다.

학년 반 번호 _____ 이름 _____

STEP 1. 텍스트가 숫자가 되기까지 (Phase 15)

앱 페이즈 15에서 ① 글자 → 코드포인트 → ② 직접 입력 → ③ 한글·영어·이모지 비교를 따라가며 숫자 변환 과정을 관찰합니다.

코드포인트와 UTF-8

- 모든 글자는 **유니코드 코드포인트**(고유 번호)를 가짐. 예: 'A' = 65, '안' = 50504(U+C548).
- 저장·전송할 때 **UTF-8** 규칙으로 1~4 바이트의 0/1 묶음으로 펼쳐짐.
- 영어 알파벳 = 1바이트, 한글 = 3바이트, 이모지 = 4바이트가 일반적.

1–1 "안녕하세요" 코드포인트 기록

앱이 보여주는 다섯 글자의 코드포인트와 UTF-8 바이트 수를 표에 옮겨 적으세요.

글자	안	녕	하	세	요
코드포인트(10진)					
UTF-8 바이트 수					

1–2 한·영·이모지 길이 비교

문자열	글자 수	UTF-8 총 바이트
"AI"		
"인공지능"		
"🤖🧠"		

한글이나 이모지가 영어보다 바이트를 더 많이 쓰는 이유는 무엇일까요?

STEP 2. 토큰 — 단어보다 작고 글자보다 큰 조각 (Phase 16)

앱 페이즈 16에서 ① 토크나이저 시연 → ② BPE 미니 시뮬레이터 → ③ 영어 vs 한글 비교를 진행.

토큰(token)

모델은 글자도 단어도 아닌 토큰 단위로 문장을 받음. 자주 등장하는 조합은 **한 토큰**으로, 드물게 등장하는 조합은 **글자 단위**까지 쪼개짐.
▸ 토큰화 규칙: **BPE**(Byte-Pair Encoding) 등 — "가장 자주 붙어 다니는 두 조각을 한 토큰으로 합친다"를 반복.

2-1 토큰 수 비교 — 앱 시연 결과 기록

예문	글자 수	토큰 수
Hello world		
Tokenization is fun		
안녕하세요		
공부했어요		
GPT는 한글을 잘게 쪼개요		

2-2 BPE 미니 시뮬레이션 따라하기

초기 어휘는 글자 단위. 가장 자주 붙어 다니는 두 조각을 합쳐가며 토큰을 만들어 갑니다.

예시 코퍼스 "low", "lower", "newest", "widest" (각각 5번씩)

아래 단계에서 다음에 합칠 두 조각을 적어보세요. (앱 시뮬레이터의 진행 순서를 따라가며)

단계	현재 빈도 1위 페어	합친 결과
1		
2		
3		

2-3 한글이 토큰을 더 많이 차지하는 이유

"공부했어요"가 1글자처럼 보이지만 토큰이 5~8개로 쪼개지는 이유를 학습 코퍼스 관점에서 설명하세요.

실용 영향 같은 분량의 글이라도 한글이 영어보다 **토큰 단가**가 더 비싸다 → API 사용료 차이의 원인.

STEP 3. 원-핫에서 임베딩으로 (Phase 17)

앱 페이즈 17에서 ① 원-핫의 한계 → ② 임베딩 행렬 W → ③ 직접 만져보기를 진행.

원-핫 vs 임베딩

- **원-핫**: 어휘가 V개면 V차원 벡터. 단어 i는 i번째 자리만 1, 나머지 0.
- **임베딩**: 학습되는 작은 차원(예: 3, 50, 768) 벡터. **의미가 비슷한 단어가 가까이** 배치되도록 학습됨.

3-1 원-핫의 한계 — 직접 계산

어휘 V = 6일 때 두 단어의 원-핫 벡터를 적고, 코사인 유사도를 계산해보세요.

```
"고양이" = ( _____, _____, _____, _____, _____, _____ )
"강아지" = ( _____, _____, _____, _____, _____, _____ )
cos = (내적) / (크기 × 크기) = _____
```

"고양이"와 "강아지"는 사람 보기에 매우 비슷한 단어인데, 원-핫에서는 모든 다른 단어 쌍과 똑같이 cos = 0이 됩니다. 이 현상이 왜 문제가 될까요?

3-2 임베딩 — 차원 줄이고 의미 담기

앱은 V = 6 단어를 3차원 벡터로 임베딩한 결과를 보여줍니다. 그림에서 가까이 모인 단어 그룹과 멀리 떨어진 그룹을 적어보세요.

가까이 모인 그룹	왜 가깝게 학습됐을까 (의미적으로)

3-3 임베딩 행렬 W의 정체

앱에서 단어 → 임베딩 벡터로 가는 과정은 결국 **원-핫 × W**의 곱셈입니다. W의 크기는 V × d (어휘 × 임베딩 차원).

```
어휘 V = 6, 임베딩 차원 d = 3일 때 W의 크기 = _____ × _____ = 학습할 숫자 _____개
```

"원-핫 × W = 임베딩"의 곱셈 결과는 사실상 W의 어떤 부분을 그대로 꺼내오는 것과 같다. 어떤 부분인가?

STEP 4. Word2Vec 미니 — 의미 연산 (Phase 18)

앱 페이즈 18에서 ① 학습 → ② 잠재 공간 → ③ 의미 산수를 진행. 영어/한국어 코퍼스 둘 다 시도 권장.

Word2Vec의 발견

단어 임베딩 공간에서 **벡터 덧셈/뺄셈이 의미 연산이 된다**는 사실.

▸ `vec(king) − vec(man) + vec(woman) ≈ vec(queen)`

→ 학습된 임베딩 공간이 "왕족 vs 일반 / 남성 vs 여성" 같은 **의미의 축**을 자동으로 잡았다는 증거.

4-1 학습 후 의미 산수 결과 기록

식	가장 가까운 단어 (앱 출력)	기대 답
king – man + woman		queen
prince – man + woman		princess
왕 – 남자 + 여자		여왕
아빠 – 남자 + 여자		엄마

4-2 왜 이런 산수가 가능한가

"king"과 "queen"은 의미상 한 가지(성별)만 다르다. 임베딩 공간에서 두 단어를 잇는 화살표가 의미하는 것이 무엇인지 적어보세요.

"father → mother"의 화살표와 "king → queen"의 화살표는 임베딩 공간에서 어떻게 비슷할까?

4-3 실패 사례도 적어보기

앱에서 결과가 "기대 답과 다른" 경우도 있나요? 한 가지 사례와 그 이유 추측을 적어보세요.

STEP 5. 6부 전반 마무리

정리 한 줄 정리 빈칸

```
글자는 (  ____________  )와 (  ____________  )를 거쳐 0과 1이 되고,
문장은 (  ____________  ) 단위로 쪼개진 뒤 (  ____________  ) 벡터로 변환된다.
의미가 비슷한 단어가 (  ____________  ) 곳에 모이도록 학습된 결과,
벡터의 (  ____________  )과 (  ____________  )이 의미의 연산이 된다.
```

서술 **친구에게 설명하기**

"GPT는 글자가 아니라 토큰을 본다. 그리고 각 토큰을 임베딩 벡터로 바꾼다." 이 두 문장을 6부 전반에서 배운 용어를 사용해 5~7문장으로 풀어 설명하세요.